Лабораторні 8, 9, 10 — пояснення простими словами

для захисту · Бабин Богдан Олегович, 443-4

Як три лаби пов’язані між собою

Усі три лаби — про одне: **як отримати хорошу нейромережу, коли своїх фото мало**. У лабі 7 ми вчили свою CNN з нуля на 17,5 тис. фото і отримали ~85%. Далі кожна лаба додає по одній техніці:

Лаба	Техніка	Що робимо одним реченням	Найкращий результат
7	CNN з нуля	мережа вчиться бачити все сама	84,75%
8	Transfer Learning	беремо «очі» мережі з ImageNet, вчимо лише класифікатор	97,56% (VGG16)
9	Fine Tuning + підбір гіперпараметрів	трохи донавчаємо останні блоки «очей»	98,00% (InceptionV3)
10	Data Augmentation + мультиклас	штучно множимо фото поворотами/зсувами; 3 і 24 класи	див. звіт лаби 10

Лабораторна 8. Перенесення навчання (Transfer Learning)

Ідея

VGG16, InceptionV3, ResNet50 — це мережі, які хтось уже навчив на ImageNet (1,2 млн фото, 1000 класів). Їхні згорткові шари вміють виділяти ознаки: перші — краї й кольори, середні — текстури, останні — частини об’єктів (вуха, очі, шерсть). Це як найняти досвідченого фотографа замість того, щоб вчити новачка: «очі» в нього вже є, йому треба лише пояснити, чим кіт відрізняється від собаки.

Що робили по завданнях

- **1. Дані.** Той самий скрипт з лаби 7: відкидаємо биті фото, ділимо 70/15/15. Взяли по 3000 фото класу — більше не треба, бо вчимо лише маленьку частину мережі.
- **2. Завантажили VGG16** з *include_top=False* (відрізали її класифікатор на 1000 класів) і **заморозили** (*trainable=False*) — ваги ImageNet не змінюються.
- **3. Додали свій класифікатор:** Flatten (для VGG16) або GlobalAveragePooling (для Inception/ResNet, бо там 2048 каналів і Flatten дав би десятки млн параметрів) → Dense 256 → Dropout → Dense 1 sigmoid.
- **4. Досліджували Dropout** 0 / 0,3 / 0,5 / 0,7. Без Dropout train 99,8%, val 96,3%, а втрати на перевірці ростуть — перенавчання. Що більший Dropout, то менший розрив train−val.
- **5. Повторили для InceptionV3 і ResNet50**, розпізнали свої 20 фото + 6 «в траві».
- **6. Зберегли** найкращі моделі у *.keras*.

Чим відрізняється від лаби 7

У лабі 7 вчилися **всі** ~1,2 млн параметрів з нуля, тут — лише 0,5–2 млн параметрів класифікатора, а 15–24 млн «очей» заморожені. Тому: точність 97% замість 85%, навчання 2 хв замість 6+, фото потрібно в 4 рази менше, і на фото «кіт у траві» мережа вже не плутається (5–6/6 замість 3/6), бо дивиться на тварину, а не на фон.

Важливі деталі коду

- `preprocess_input` замість `rescale=1/255` — фото треба подати так само, як при навчанні на ImageNet (кожна мережа має свою нормалізацію).
- `Adam(1e-4)` — мала швидкість, бо класифікатор сидить на вже готових ознаках.

Лабораторна 9. Тонке налаштування (Fine Tuning)

Ідея

У лабі 8 «очі» були повністю заморожені. Але останні блоки налаштовані під 1000 класів ImageNet, а не саме під котів/собак. Fine tuning — це **розморозити 1–2 останні блоки і трохи їх донавчити** з дуже малою швидкістю (1e-5), щоб підлаштувати ознаки під нашу задачу. Аналогія: фотограф уже вміє знімати, ми лише трохи підкручуємо налаштування під конкретну зйомку.

Що робили

- **Підбір гіперпараметрів** (Exercises): нейрони Dense 32–1024, Dropout 0,25–0,8, додатковий Dense, батч 25–125, оптимізатори sgd/adam/rmsprop, lr 1e-3…1e-6, епохи 5–25 — ~30 експериментів на мережу.
- **Хитрість для швидкості:** поки згорткова частина заморожена, її вихід для кожного фото однаковий — порахували ознаки **один раз** і вчили класифікатор на них. 30 експериментів за 4 хв замість години.
- **Fine tuning:** 1 блок з lr 1e-5, 1e-4, 1e-6 і 2 блоки з lr 1e-5. Порядок обов’язковий: спочатку навчити класифікатор, потім розморожувати (інакше великі випадкові градієнти зламають ваги ImageNet). BatchNormalization лишаємо замороженими.

Що вийшло і чому

- Найкраща — InceptionV3 з підібраними гіперпараметрами, **98,00%**. Найбільше впливає швидкість навчання: 1e-4 найкраща, 1e-6 — мережа майже не вчиться.
- Fine tuning допоміг лише VGG16 (+0,8%). Для Inception/ResNet — ні: 4200 фото замало для 5–12 млн розморожених параметрів, вони перенавчаються за 1–3 епохи. lr 1e-4 «ламає» ваги, 2 блоки — перенавчання.
- VGG16 на тесті після підбору гірша, ніж у лабі 8 — бо обирали за перевірочними даними, де варіанти відрізнялись на 3–5 фото з 900. Це чесний результат, він показує межі підбору.

Чим відрізняється від лаби 8

Лаба 8 — заморожено **все**, досліджуємо один параметр (Dropout). Лаба 9 — розморожуємо частину «очей» і систематично підбираємо всі гіперпараметри.

Лабораторна 10. Доповнення даних (Data Augmentation) і мультикласова класифікація

Ідея

Data Augmentation — **штучно збільшуємо навчальний набір**: щоепохи кожне фото випадково повертається (до 40°), зсувається, масштабується, віддзеркалюється. Мережа ніколи не бачить те саме фото двічі однаково, тому не може його «завчити» — менше перенавчання. Важливо: доповнюємо **лише навчальні** дані, перевірочні й тестові — ні (окремий генератор без аугментації), бо оцінювати треба на справжніх фото.

Завдання 1. Коні/люди, InceptionV3, Functional API

Писали код лише між START/END CODE HERE. Беремо InceptionV3 до шару **mixed7** (7×7×768), Flatten → Dense 1024 → Dropout 0,2 → sigmoid. **Functional API** (`Model(inputs=..., outputs=...)`) замість Sequential — бо «відрізаємо» мережу посередині і приклеюємо свій вихід до проміжного шару. Колбек зупиняє навчання на 99,9% — спрацював на 2-й епосі (val 99,6%). Технічно: увімкнули Keras 2 (tf_keras), бо ноутбук використовує *output_shape*, якого немає в Keras 3.

Завдання 2. Тестові фото, 300×300, аугментація, fine tuning

- **Нові фото:** 14 справжніх фото коней і людей + 3 «обманні» (людина з конем). 14/14 — хоча навчальні фото там **згенеровані 3D-моделі**! Це сила перенесення навчання: InceptionV3 бачила справжніх коней на ImageNet.
- **300×300:** карта ознак 17×17 замість 7×7 → параметрів у 6 разів більше (227 млн проти 38 млн), val впала до 97%, нові фото 13/14 — гірше: більше параметрів на ті самі 1027 фото = більше перенавчання.
- **Аугментація:** вчилось довше (7 епох замість 2), val коливається 98–100%; на нових фото так само 14/14. Набір дуже простий (3D-моделі на однотонному фоні), тому аугментація майже нічого не додає — перевірочні фото схожі на навчальні.
- **Fine tuning** одного шару: зупинилось одразу (точність уже 100%), результат як без нього. «Обманні» фото: модель плутає «жінку з конем» — на фото є і те, і те.

Завдання 3. Коти/собаки 2000/500/500: VGG16, InceptionV3, ResNet50 + аугментація + fine tuning

- Скрипт з лаби 7 змінили: рівно 2000/500/500 без повторів (4000 навчальних фото — у 4 рази менше, ніж у лабі 7).
- **Без аугментації** VGG16 завчила навчальні фото: train 99,7%, а втрати на перевірці 0,5 — перенавчання. Тест 96,4%.
- **З аугментацією** train *нижча* за val (95,5% проти 97,6%) — фото щоразу інші, завчити не вийде. Втрати на перевірці впали в 4 рази, тест 96,8%. Але епоха 35 с замість 14 с.
- **+ fine tuning** 1 блоку — 97,4%, 2 блоків — **97,6%** (найкраще). Слабша аугментація — гірше (97,2%, знову розрив).
- InceptionV3: аугментація дала +1,2% (96,3 → 97,5%). ResNet50: ніде не перенавчалась (класифікатор лише 65 тис. параметрів), тому аугментація нічого не додала — 97,4%.

- 20 нових фото котів/собак з інтернету (свої фото не вдалося завантажити в Colab): VGG16 19/20, InceptionV3 20/20, ResNet50 19/20.

Завдання 4. Камінь-ножиці-папір (3 класи)

Різниця з бінарною класифікацією: на виході **3 нейрони + softmax** (ймовірності, що в сумі дають 1) замість 1 нейрона + sigmoid, і втрати *categorical_crossentropy* замість *binary*. Результат: val **98,9%**, фото з лекції 9/9. Справжні фото рук з інтернету — лише **8/12** (кулаки називала ножицями): вчили з нуля на 3D-рендерах вертикальної руки на білому фоні, а на фото рука горизонтально, з передпліччям. Експеримент з фото предметів не виконано — файли не вдалося завантажити в Colab (можна дозавантажити в папку objects і запустити останні комірки).

Завдання 5. Мова жестів (Sign Language MNIST, CSV)

- Дані не у вигляді картинок, а в CSV: рядок = мітка + 784 пікселі. Функція *parse_data_from_input* читає CSV, пропускає заголовок і робить масиви 28×28 типу float64.
- Чорно-білі фото → додаємо вимір каналу: (27455, 28, 28) → (27455, 28, 28, 1). Генератор — *flow* (з масивів), а не *flow_from_directory*.
- 26 нейронів softmax, втрати *sparse_categorical_crossentropy* (мітки — числа, не one-hot). Класів реально 24: J і Z показуються рухом, їх у наборі немає.
- Аугментація без віддзеркалення — дзеркальний жест може бути іншою літерою. Результат: **train 99,07%, val 99,23%** (треба було >99% і >95%).

Чим лаба 10 відрізняється від 8–9

Лаби 8–9 боролися з перенавчанням через Dropout і заморожування. Лаба 10 бореться **через дані** — робить їх різноманітнішими. Плюс уперше більше двох класів (3 і 24) і дані з CSV замість папок.

Можливі питання на захисті

— Що таке перенесення навчання?

Беремо мережу, навчену на великому наборі (ImageNet), її згорткові шари використовуємо як готовий виділювач ознак, а вчимо лише свій класифікатор.

— Навіщо include_top=False?

Щоб відрізати класифікатор на 1000 класів ImageNet — нам потрібні лише 2 (або 3, 24) класи.

— Чим fine tuning відрізняється від transfer learning?

У transfer learning згорткова частина повністю заморожена. У fine tuning розморожуємо останні блоки і донавчаємо їх з дуже малою швидкістю.

— Чому не можна одразу розморозити і вчити?

Класифікатор спочатку випадковий, його великі градієнти зламають ваги ImageNet. Спочатку навчаємо класифікатор, потім розморожуємо.

— Чому швидкість для fine tuning 1e-5?

Щоб ваги змінювались трохи — підлаштування, а не перенавчання з нуля. 1e-4 вже ламала ваги (точність падала на 1-й епосі).

— Що дає Dropout?

На кожному кроці випадково вимикає частину нейронів — мережа не може покладатися на окремі нейрони, менше перенавчання. На перевірці працюють усі.

— Що таке Data Augmentation і чому лише для навчальних даних?

Випадкові повороти/зсуви/масштаб/віддзеркалення навчальних фото — мережа бачить нові варіанти, менше завчує. Перевіряємо на справжніх фото, тому їх не змінюємо.

— Чому з аугментацією навчальна точність нижча/росте повільніше?

Задача складніша — фото щоразу інші. Зате розрив train−val менший.

— Чим Functional API відрізняється від Sequential?

Sequential — лише шари підряд. Functional — довільний граф: можна взяти вихід із середини іншої моделі (mixed7), кілька входів/виходів.

— sigmoid чи softmax?

sigmoid + binary_crossentropy — 2 класи, 1 вихід. softmax + categorical_crossentropy — N класів, N виходів, сума ймовірностей = 1.

— sparse_categorical чи categorical?

sparse — мітки цілі числа (0…25), categorical — one-hot вектори. Генератор flow_from_directory з class_mode="categorical" дає one-hot.

— Чому 300×300 гірше за 150×150 у коней/людей?

Більша карта ознак → у 6 разів більше параметрів у Dense при тих самих 1027 фото → швидше перенавчання, повільніше навчання.

— Чому GlobalAveragePooling, а не Flatten для InceptionV3/ResNet?

2048 каналів: Flatten дав би десятки млн параметрів. GAP усереднює кожну карту до одного числа — 2048 входів.

— Чому preprocess_input, а не rescale?

Мережа навчалась на ImageNet з певною нормалізацією (VGG — віднімання середнього, Inception — до [-1, 1]). Треба подавати так само.

— Чому transfer learning краще за свою CNN з лаби 7?

Ознаки вивчені на 1,2 млн різних фото — мережа розпізнає саму тварину, а не фон. Своя CNN на 17,5 тис. фото частково вчила фон (коти в траві → «собака»).